# LAPORAN PRAKTIKUM 6
# INHERITANCE

**Disusun Oleh:**
**KHOHAR MUHAMAD FATAHURROHMAN**
**254107020021**
**Kelas TI 2B**




**PROGRAM STUDI TEKNIK INFORMATIKA**
**JURUSAN TEKNOLOGI INFORMASI**
**POLITEKNIK NEGERI MALANG**
**Oktober 2026**

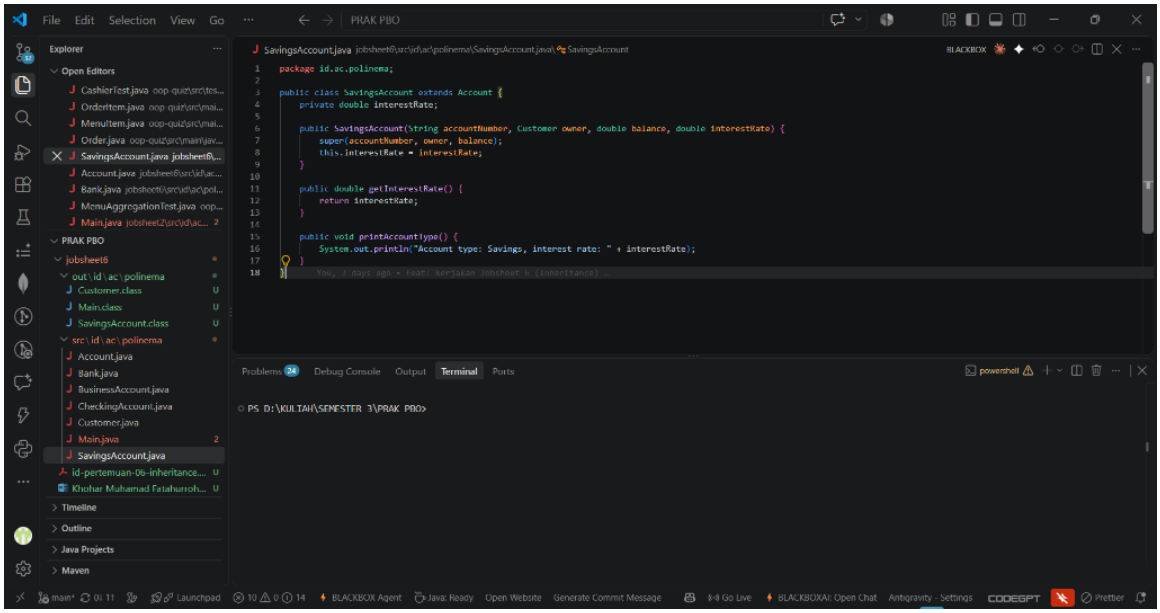
## Langkah Percobaan

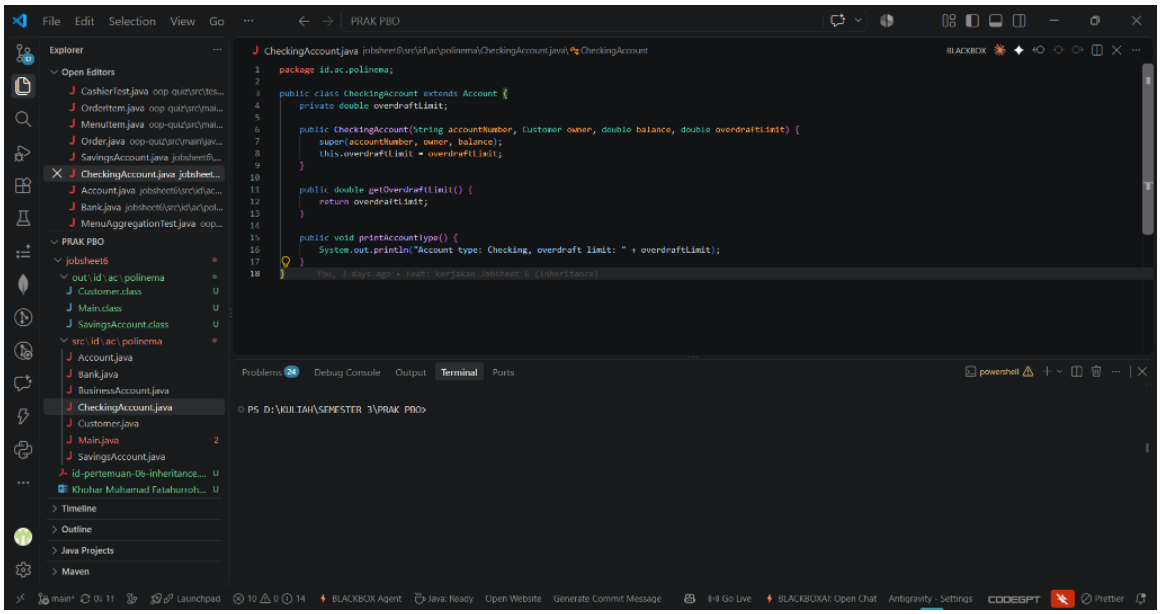
### a. Struktur Project

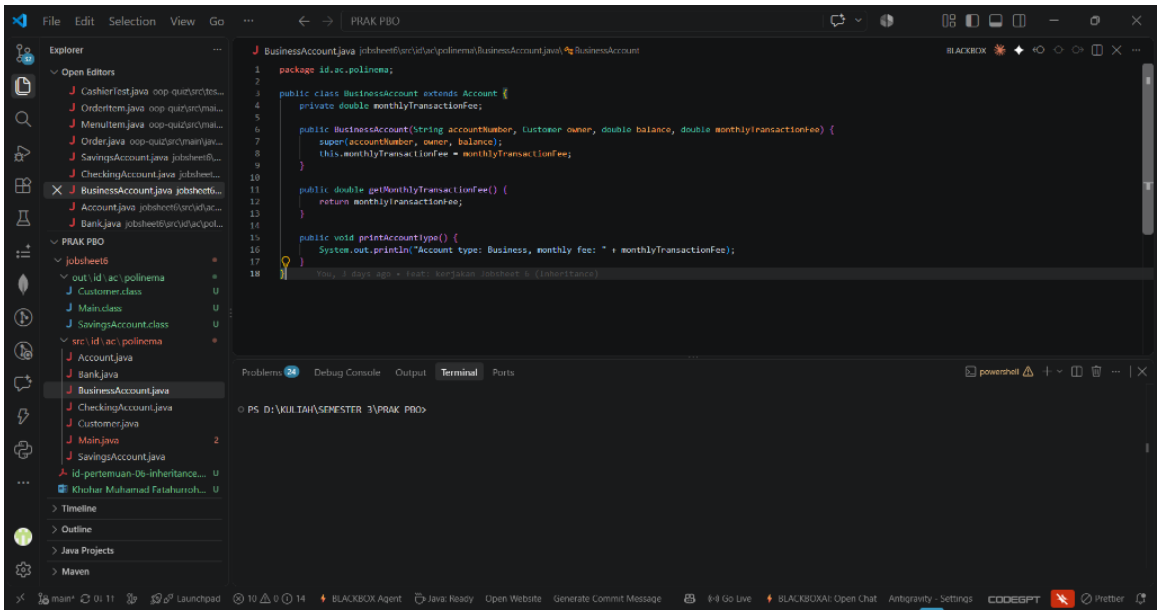
Proyek ini bernama bank-mini (lanjutan pertemuan 4) dan diorganisasikan menggunakan struktur package id.ac.polinema. Package ini berfungsi mengelompokkan kelas-kelas Java yang berkaitan, menghindari konflik nama kelas (name collision), serta mempermudah manajemen access modifier. Di dalam package ini terdapat tambahan tiga subclass baru dari Account:

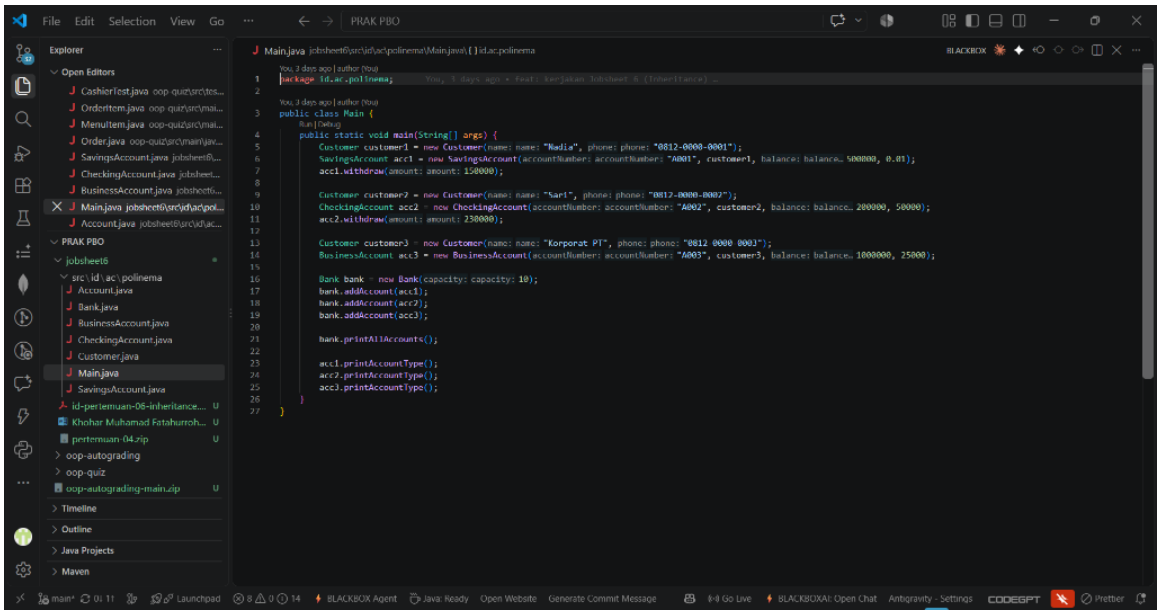
1. SavingsAccount.java: Bertindak sebagai subclass dari Account dengan penambahan atribut interestRate.
2. CheckingAccount.java: Bertindak sebagai subclass dari Account dengan penambahan atribut overdraftLimit.
3. BusinessAccount.java: Merupakan tugas mandiri, bertindak sebagai subclass dari Account dengan penambahan atribut monthlyTransactionFee.

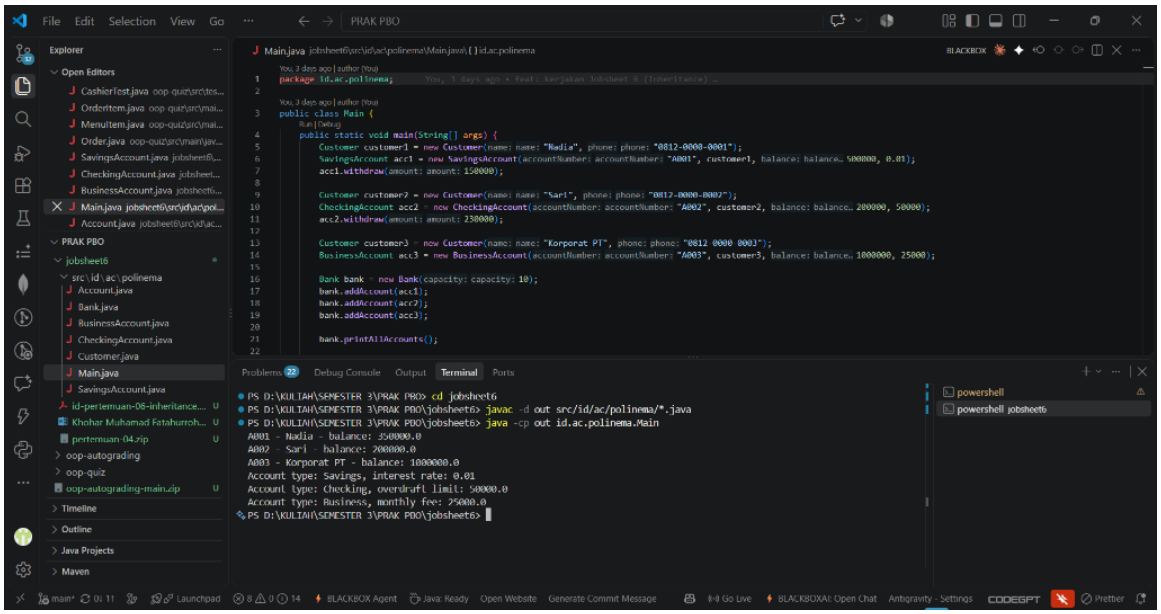
### b. Implementasi & Menjalankan Program




File Edit Selection View Go
PRAK PBO
CheckingAccount.java jobsheet6\src\id\ac\polinema\CheckingAccount.java\CheckingAccount
BLACKBOX
Explorer
Open Editors
CashierTest.java oop-quiz\src\tes...
OrderItem.java oop-quiz\src\mai...
MenuItem.java oop-quiz\src\mai...
Order.java oop-quiz\src\main\jav...
SavingsAccount.java jobsheet6\...
CheckingAccount.java jobsheet...
Account.java jobsheet6\src\id\ac...
Bank.java jobsheet6\src\id\ac\pol...
MenuAggregationTest.java oop...
PRAK PBO
jobsheet6
out\id\ac\polinema
Customer.class
Main.class
SavingsAccount.class
src\id\ac\polinema
Account.java
Bank.java
BusinessAccount.java
CheckingAccount.java
Customer.java
Main.java
SavingsAccount.java
id-pertemuan-06-inheritance....
Khohar Muhamad Fatahurroh...
Timeline
Outline
Java Projects
Maven
1 package id.ac.polinema;
2
3 public class CheckingAccount extends Account {
4 private double overdraftLimit;
5
6 public CheckingAccount(String accountNumber, Customer owner, double balance, double overdraftLimit) {
7 super(accountNumber, owner, balance);
8 this.overdraftLimit = overdraftLimit;
9 }
10
11 public double getOverdraftLimit() {
12 return overdraftLimit;
13 }
14
15 public void printAccountType() {
16 System.out.println("Account type: Checking, overdraft limit: " + overdraftLimit);
17 }
18 }
You, 3 days ago • feat: kerjakan jobsheet 6 (inheritance)
Problems 24
Debug Console
Output
Terminal
Ports
powershell
PS D:\KULIAH\SEMESTER 3\PRAK PBO>
main*
Launchpad
BLACKBOX Agent
Java: Ready
Open Website
Generate Commit Message
Go Live
BLACKBOXAI: Open Chat
Antigravity - Settings
CODEGPT
Prettier


File Edit Selection View Go
PRAK PBO
BusinessAccount.java jobsheet6\src\id\ac\polinema\BusinessAccount.java\BusinessAccount
BLACKBOX
Explorer
Open Editors
CashierTest.java oop-quiz\src\tes...
OrderItem.java oop-quiz\src\mai...
MenuItem.java oop-quiz\src\mai...
Order.java oop-quiz\src\main\jav...
SavingsAccount.java jobsheet6\...
CheckingAccount.java jobsheet...
BusinessAccount.java jobsheet6...
Account.java jobsheet6\src\id\ac...
Bank.java jobsheet6\src\id\ac\pol...
PRAK PBO
jobsheet6
out\id\ac\polinema
Customer.class
Main.class
SavingsAccount.class
src\id\ac\polinema
Account.java
Bank.java
BusinessAccount.java
CheckingAccount.java
Customer.java
Main.java
SavingsAccount.java
id-pertemuan-06-inheritance....
Khohar Muhamad Fatahurroh...
Timeline
Outline
Java Projects
Maven
1 package id.ac.polinema;
2
3 public class BusinessAccount extends Account {
4 private double monthlyTransactionFee;
5
6 public BusinessAccount(String accountNumber, Customer owner, double balance, double monthlyTransactionFee) {
7 super(accountNumber, owner, balance);
8 this.monthlyTransactionFee = monthlyTransactionFee;
9 }
10
11 public double getMonthlyTransactionFee() {
12 return monthlyTransactionFee;
13 }
14
15 public void printAccountType() {
16 System.out.println("Account type: Business, monthly fee: " + monthlyTransactionFee);
17 }
18 }
You, 3 days ago • feat: kerjakan jobsheet 6 (inheritance)
Problems 24
Debug Console
Output
Terminal
Ports
powershell
PS D:\KULIAH\SEMESTER 3\PRAK PBO>
main*
Launchpad
BLACKBOX Agent
Java: Ready
Open Website
Generate Commit Message
Go Live
BLACKBOXAI: Open Chat
Antigravity - Settings
CODEGPT
Prettier





Penjelasan Kode, Konsep OOP, dan Kendala:

Pada kode di atas, kita menerapkan prinsip Inheritance (Pewarisan). Kelas SavingsAccount, CheckingAccount, dan BusinessAccount mewarisi atribut dan metode dari kelas induknya yaitu Account dengan menggunakan keyword extends. Konstruktor subclass menggunakan fungsi super(...) untuk memanggil konstruktor

dari superclass guna menginisialisasi atribut bawaan, lalu menambahkan atribut spesifik di masing-masing subclass.

**Hasil & Pembahasan**

Berdasarkan hasil akhir dari program ini, penerapan Inheritance berjalan dengan sangat baik. Penggunaan extends memungkinkan terjadinya reusability kode tanpa perlu menulis ulang atribut seperti accountNumber, owner, maupun method deposit() dan withdraw(). Polimorfisme mulai terlihat pada penggunaan array of Account di dalam class Bank yang mampu menyimpan objek-objek subclass-nya secara bersamaan dan mencetak saldo akhirnya melalui perulangan yang rapi.

**Jawaban Tugas Mandiri:**

1. BusinessAccount sudah diimplementasikan (lihat gambar di bagian implementasi).
2. Mengapa Bank.printAllAccounts() belum bisa menampilkan info khusus tiap jenis rekening? Jawaban: Hal ini terjadi karena class Bank menyimpan semua rekening di dalam array bertipe Account. Di kelas Account (superclass), tidak terdapat deklarasi method printAccountType(). Java hanya mengenali method yang ada di Account. Agar method khusus subclass bisa dipanggil secara otomatis, kita perlu melakukan method overriding, misalnya dengan merombak method printInfo() pada masing-masing subclass untuk menimpa perilaku default dari superclass.
3. Apakah SavingsAccount sebaiknya dipecah lagi menjadi subclass? Jawaban: Ya, jika logika perhitungannya mulai bervariasi. Contoh: kita bisa membuat class TieredSavingsAccount extends SavingsAccount dengan tambahan atribut double[] interestThresholds untuk menghitung bunga berjenjang sesuai

batas saldo. Namun, untuk skala Bank Mini saat ini, pemecahan itu belum terlalu mendesak.

## Kesimpulan

Dari praktikum Jobsheet 6 ini, dapat disimpulkan bahwa prinsip Inheritance sangat penting dalam OOP untuk mencegah redundansi kode (pengulangan penulisan kode yang sama). Dengan menggunakan keyword extends dan method super(...), kita dapat dengan mudah mengembangkan blueprint kelas dasar menjadi tipe-tipe spesifik (seperti SavingsAccount atau CheckingAccount) sambil tetap menjaga fleksibilitas dan pemusatan perilaku umum pada superclass.

## Lampiran

Link Repositori GitHub untuk Source Code Lengkap:

https://github.com/helfa25/PRAKTIKUM-PBO.git